

University of Pisa

School of Engineering

Department of Information Engineering

MSc Computer Engineering

Internet of Things

Smart HVAC:
A Climate Control System

Author:

Lorenzo Vezzani

Academic Year 2025/26

Summary

1 Introduction **2**
- 1.1 Project Overview . . . 2
- 1.2 Use case . . . 2
- 1.3 Code Repository . . . 2
- 1.4 Environment . . . 2

2 Architecture **3**
- 2.1 Sensor Node . . . 3
- 2.2 Actuator Node . . . 3
- 2.3 Border Router . . . 4
- 2.4 Cloud Application . . . 4
  - 2.4.1 Dispatcher . . . 4
  - 2.4.2 Controller . . . 4
  - 2.4.3 MQTT Client . . . 5
  - 2.4.4 Database . . . 5
  - 2.4.5 Command Line Interface . . . 5

3 Communication **7**
- 3.1 MQTT . . . 7
- 3.2 MQTT Messages . . . 7
- 3.3 Data Encoding . . . 8

4 Machine Learning **9**
- 4.1 Occupancy Detection Model . . . 9
- 4.2 Dataset . . . 9
- 4.3 Data Preprocessing . . . 9
- 4.4 Model Deployment . . . 9

5 Adaptive Mechanism **10**
- 5.1 Adaptive mechanism . . . 10
- 5.2 Evaluation . . . 10

6 Conclusion **12**
- 6.1 Limitations . . . 12
- 6.2 Future Work . . . 12

Chapter 1

Introduction

1.1 Project Overview

Smart HVAC is an IoT-based system designed to improve the energy efficiency of heating, ventilation, and air conditioning in smart buildings. The system combines environmental sensing, occupancy detection, a fall-back adaptive mechanism, and cloud-based control to dynamically manage HVAC operation according to the current conditions.

1.2 Use case

The proposed use case considers a smart building composed of multiple rooms equipped with IoT sensors and HVAC actuators. Environmental data are collected and processed to detect room occupancy and determine whether the HVAC system should be activated, while the cloud component coordinates the available energy budget among the different rooms.

1.3 Code Repository

The source code of the project is available at
`https://github.com/lorenzo-vezzani/smart-hvac-iot/`.

1.4 Environment

- **Contiki-NG:** an open-source operating system for low-power and resource-constrained IoT devices, supporting IPv6, RPL, CoAP, and MQTT. Its modular architecture facilitates the development and testing of embedded IoT applications (Oikonomou et al. 2022).[1]
- **Cooja:** the network simulator integrated with Contiki-NG, used to emulate IoT networks and evaluate sensor and actuator nodes in a controlled environment before deployment on physical hardware.
- **Tunslip:** a Contiki-NG tool that establishes a virtual SLIP connection between the border router and the host machine, enabling communication between the simulated IoT network and external cloud applications.
- **Nordic nRF52840 Dongle:** a USB development platform based on the nRF52840 SoC with an ARM Cortex-M4 processor and IEEE 802.15.4 support. It was used as physical hardware for testing the embedded components of the system.

[1] Contiki-NG: http://github.com/contiki-ng/contiki-ng/

Chapter 2

Architecture

2.1 Sensor Node

The sensor node is responsible for collecting and publishing environmental data to the MQTT broker with a sampling rate of 60 seconds. The monitored parameters include temperature, humidity, light, and CO_2. Since the system is evaluated in a simulated environment, sensor values are generated randomly while taking into account the HVAC activation state of the corresponding room, resulting in more realistic environmental variations.
Going into implementation details we define the following possible states, each one associated with a specific led configuration.

- **OFF**: the node waits for a button press to start its operation.
- **REGISTER**: the sensor initializes its MQTT client and registers itself with the MQTT communication process.
- **CONNECT**: the node waits for network connectivity and establishes a connection with the MQTT broker.
- **SUBSCRIBE**: after successfully connecting to the broker, the sensor subscribes to the MQTT topics used to receive HVAC commands.
- **GENERATE**: the sensor generates the environmental measurements, taking into account the current HVAC state of the room.
- **PUBLISH**: the generated measurements are encoded in a SenML payload and published to the MQTT broker.

Whenever a button is pressed the second time, the node will be shut down.

2.2 Actuator Node

The actuator node processes the environmental information associated with its room and uses it to predict occupancy through a machine learning model, which is described in a later section. The HVAC system is activated when the predicted occupancy is true and the measured temperature exceeds a predefined threshold. The system is switched off whenever either of these conditions is no longer satisfied. The actuator also listens to the MQTT topics used by the cloud application to dynamically modify the HVAC power level. Each change of HVAC state is reported to the MQTT broker, while the shutdown notification also includes the energy consumed during the activation period.
As well as the sensor, here's a brief description of the actuator's states:

- **OFF:** The actuator is inactive and waiting for button pressing.
- **REGISTER:** The actuator is registering with the MQTT communication system.
- **CONNECT:** The actuator is establishing a connection with the broker.
- **SUBSCRIBE:** The actuator is subscribing to the MQTT topics required for its operation.
- **SENSOR:** The actuator is waiting to receive new environmental data from the sensor.
- **INFERENCE:** The actuator is processing the data received from the sensor and performing the occupancy inference.

Whenever a button is pressed the second time, the node will be shut down.

2.3 Border Router

The RPL Border Router used in this project is the one provided by the official Contiki-NG repository. It can be found at the following link: `https://github.com/contiki-ng/contiki-ng/tree/develop/examples/rpl-border-router`.

2.4 Cloud Application

The cloud application is responsible for collecting sensor and actuator information and coordinating the operation of the different HVAC systems. It follows a modular architecture composed of a Dispatcher, a Controller, a Database, an MQTT Client, and a Command Line Interface. Incoming MQTT messages are received by the communication layer and forwarded to the components responsible for processing, control, and persistent storage.

2.4.1 Dispatcher

The Dispatcher manages incoming MQTT messages and determines how they should be processed. Depending on the type of message, it forwards the information to the Controller for decision-making, to the database component for persistent storage, or to both. This organization separates message handling from the application logic and database management.

2.4.2 Controller

The Controller implements the main control logic of the cloud application. It monitors the total power requested by the active HVAC systems and verifies that the building power consumption remains within the predefined limit. When the available power is sufficient, all active HVAC systems can operate at the HIGH level. Otherwise, a greedy allocation strategy distributes the available power proportionally according to the priority assigned to each room.

The resulting allocation is translated into three discrete operating levels: LOW, MID, and HIGH. The selected level is then communicated to the corresponding actuator through MQTT.

2.4.3 MQTT Client

The MQTT Client provides the communication interface between the cloud application and the MQTT broker. It manages the establishment and termination of the MQTT connection, handles incoming messages, and provides the functionality required to publish commands and notifications.

2.4.4 Database

The cloud application uses a relational database composed of three main tables.

Table 1: Database structure

Table	**Fields**
room	`id INT`, `name VARCHAR(100)`
measurement	`id BIGINT`, `room_id INT`, `timestamp DATETIME`, `temperature DOUBLE`, `humidity DOUBLE`, `light DOUBLE`, `co2 DOUBLE`
activation	`id BIGINT`, `room_id INT`, `start_time DATETIME`, `end_time DATETIME`, `energy_consumed_wh FLOAT`

2.4.5 Command Line Interface

The cloud application also provides a Command Line Interface (CLI) for monitoring and interacting with the system during runtime.

Table 2: Available CLI commands

Command	**Description**
`q, quit`	Terminates the application.
`h, help`	Displays the list of available commands
`s, status`	Displays the current status of all devices.
`d [-s ROOM] [-p BEGIN END]`	Displays the sensor measurements stored in the database. The `-s` option selects a specific room, while `-p` restricts the results to a specified time interval.
`ct <ROOM> <THRESHOLD>`	Changes the HVAC temperature threshold.

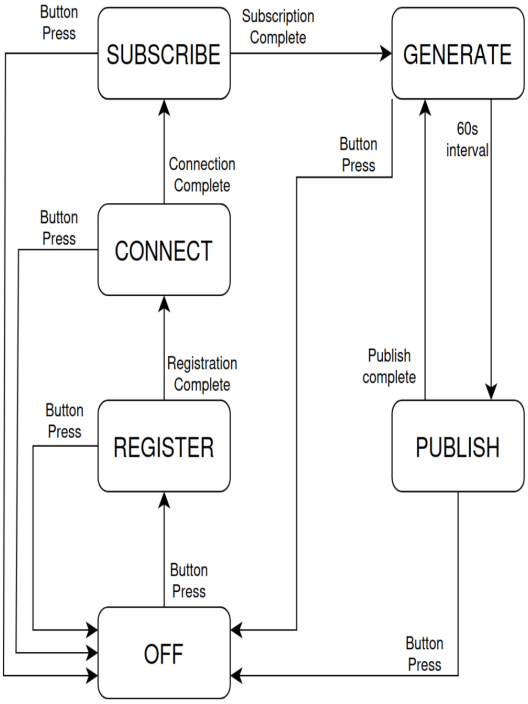


Figure 1: Sensor states.

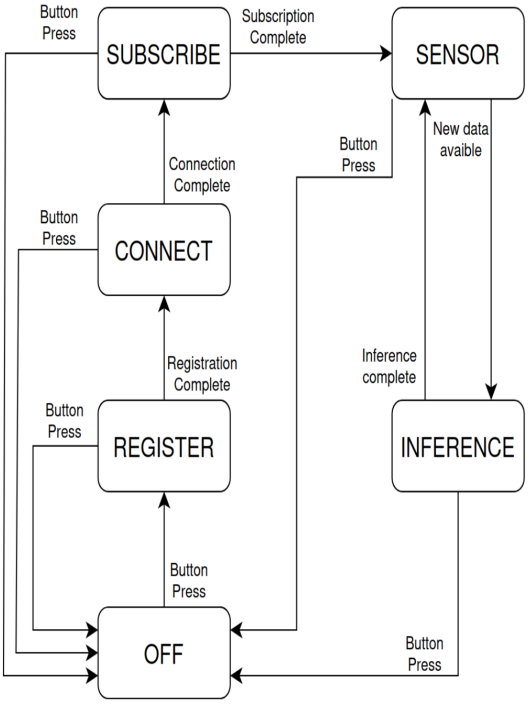


Figure 2: Actuator states.

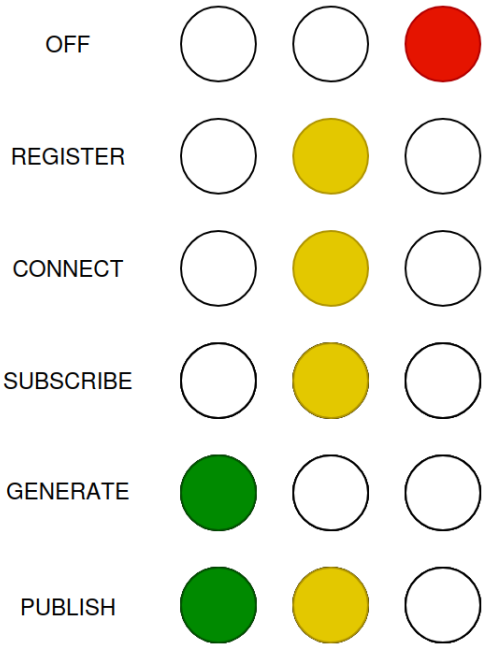


Figure 3: Sensor LEDs changes across states.

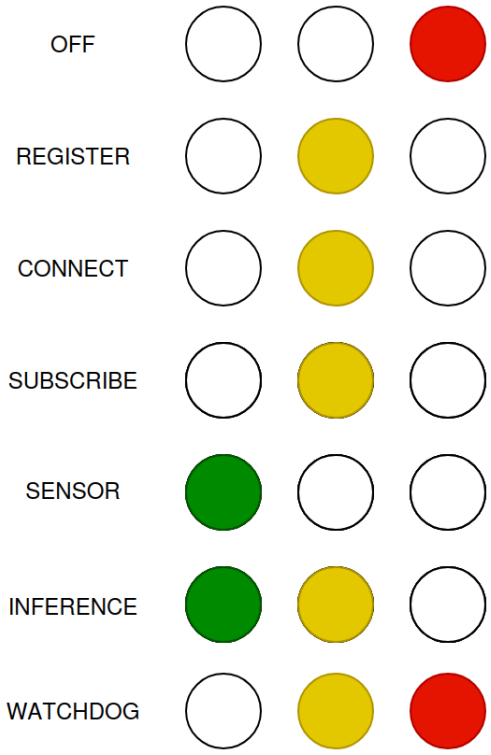


Figure 4: Actuator LEDs changes across states.

[1]Watchdog state is a state introduced for the adaptive mechanism. More details can be found in Chapter 5.

Chapter 3

Communication

3.1 MQTT

MQTT is used as the application-layer communication protocol because its lightweight publish/subscribe architecture is well suited for our purposes. The broker decouples publishers from subscribers, allowing sensors, actuators, and cloud components to communicate without requiring direct connections. Moreover, in domestic scenarios such as the one of the application described is assumable that a Wi-Fi network is present in the house. This makes MQTT perfectly suitable, since it does not require other communication technologies to be deployed.

3.2 MQTT Messages

Here's the list of the MQTT messages using in the application:

`building/<room>/telemetry`

Publisher: Sensor node

Subscriber: Actuator, Cloud application

This message periodically reports the environmental conditions of the room. The payload contains:

- `timestamp`: measurement timestamp;
- `temperature`: internal room temperature;
- `humidity`: relative humidity;
- `light`: light intensity;
- `c02`: CO_2 concentration;
- `sequence`: sequence number.

`building/<room>/hvac/start`

Publisher: Actuator node

Subscriber: Cloud application

This message notifies the cloud application that the HVAC system has been activated.

building/<room>/hvac/end

Publisher: Actuator node

Subscriber: Cloud application

This message reports the termination of an HVAC activation. The payload contains:

- `energy_consumed_wh`: energy consumed during the activation, in Wh;
- `reason`: reason for the HVAC deactivation.

building/<room>/hvac/control

Publisher: Cloud application

Subscriber: Actuator node

This message is used to dynamically modify the HVAC operating power according to the energy management decisions made by the cloud controller. The payload contains:

- `power`: HVAC operating level, `LOW`, `MID`, or `HIGH`.

building/<room>/hvac/threshold

Publisher: Cloud application

Subscriber: Actuator node

This message allows the temperature threshold used by the local HVAC control logic to be modified at runtime. The payload contains:

- `threshold`: temperature threshold above which the HVAC can be activated when occupancy is detected.

3.3 Data Encoding

The messages payload is encoded in **JSON**. In telemetry messages **SenML** representation has been used to reduce the payload size while maintaining a simple and easily parseable format. XML would have been too verbose and its main advantage, the high standardization and so interoperability, is not particulary useful for the application purposes. The simplicity and the light-weightness of JSON are the perfect combination for the application's purposes.

Chapter 4

Machine Learning

4.1 Occupancy Detection Model

The occupancy detection task is formulated as a binary classification problem. A small Deep Neural Network (DNN) is used to classify a room as either occupied or unoccupied based on four environmental measurements: Temperature, Humidity, Light, and CO_2. The DNN consists of two fully connected hidden layers with 16 and 8 ReLU neurons, followed by a single sigmoid output.

4.2 Dataset

The model is trained on a dataset containing environmental measurements and the corresponding occupancy state. The four input features are Temperature, Humidity, Light, and CO_2, while Occupancy is used as the target variable.
The dataset is available at: `https://www.kaggle.com/datasets/pooriamst/occupancy-detection`

4.3 Data Preprocessing

Since the dataset contains time-dependent measurements, the data is split chronologically rather than using random sampling. The first 70% of the samples is used as the development set, while the final 30% is kept as a completely unseen test set. The development set is further divided chronologically into 85% training data and 15% validation data.
The input features are standardized using a *StandardScaler*, which is fitted exclusively on the training set and subsequently applied to the validation and test sets. During training, early stopping monitors the validation loss to prevent overfitting.

4.4 Model Deployment

After training, the model is evaluated exclusively on the unseen test set using accuracy, precision, recall, F1-score, ROC-AUC, and a conf. matrix. Finally, the trained model is converted into a C representation using *emlearn*, generating a header file that can be directly integrated into the actuator firmware.
The resulting metrics and evaluation plots are available at: `https://github.com/lorenzo-vezzani/smart-hvac-iot/tree/main/ml`.

Chapter 5

Adaptive Mechanism

5.1 Adaptive mechanism

The HVAC system implements an adaptive mechanism based on the freshness of sensor measurements. Every actuator maintains the relative sensor state associated with one of three states:

- **Fresh**: the measurement is within the expected sampling interval and is considered representative of the current room conditions;
- **Stale**: no new measurement has been received for a limited period, but the last measurement can still be considered a reasonable approximation of the current conditions;
- **Invalid**: the measurement is too old to be reliably used for HVAC control.

With a sensor sampling rate of $60\,\mathrm{s}$, a measurement is considered stale after 1.5 sampling intervals and invalid after 3 intervals. Consequently, if no valid measurement is received for $180\,\mathrm{s}$, the actuator activates the HVAC fallback mechanism, turning the system off if it is currently running. To implement this mechanism, a new actuator state, **watchdog**, has been introduced. Every $60\,\mathrm{s}$, the actuator enters the watchdog state and compares the current time with the timestamp of the last received measurement.
This behaviour is motivated by the thermal inertia of the environment (i.e. room measurements changes relatively slowly). Consequently, temporarily missing one or two measurements does not necessarily indicate a significant change in the environmental conditions. Delaying the fallback therefore avoids unnecessarily reactive behaviour while still guaranteeing autonomous operation when the available sensor information becomes too old.

5.2 Evaluation

The adaptive mechanism is evaluated[2] by varying the packet loss rate and comparing its behaviour with a non-adaptive configuration. The metrics are comfort level and energy consumption. For each test, the comfort level is computed from the temperature values of the sensor measurements received during the test:

$$C = \frac{1}{N} \sum_{i=1}^{N} \left| T_i - T_{\text{ideal}} \right|, \tag{1}$$

[2]The evluation has been conducted by using a different version of the application, explained in the appendix

where T_i is the temperature contained in a received measurement and T_{ideal}[3] is the ideal temperature.
Energy consumption is obtained from the measurements reported by the actuator at the end of each HVAC activation. The energy value associated with a test is the average consumption across its completed activations.
Figures 5 and 6 show the comfort level and energy consumption, respectively, as the packet loss rate increases, comparing the adaptive and non-adaptive configurations.
As we can see, the energy savings increase with the packet loss percentage. This was expected, since when a large number of consecutive packets are lost while an HVAC system is active, the actuator may not receive the measurements required to switch it off, causing it to remain active for longer and consequently wasting more energy.
On the other hand, the comfort level is marginally affected by the fallback mechanism. Therefore, the proposed mechanism can be applied without significantly compromising the user's comfort.



Figure 5: Energy savings achieved by the adaptive mechanism.

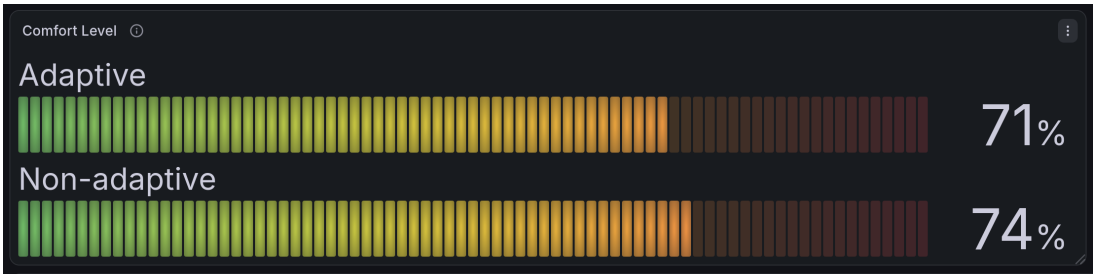


Figure 6: Comfort level comparison between adaptive and non-adaptive modes.

[3]For the sake of simplicity, T_{ideal} is considered 21.0 °C for every room.

Chapter 6

Conclusion

6.1 Limitations

The main limitation of the presented application is its static configuration. In a real-world scenario, where topology changes such as the introduction of a new room, sensor, or actuator can occur, a more automated configuration mechanism would be preferable.
Moreover, actuator failures are currently undetected. This can be problematic from an energy-management perspective, since a failed HVAC actuator may remain active without the cloud being aware of its actual state.

6.2 Future Work

The cloud infrastructure could be exploited to provide dynamic configuration, allowing rooms and devices to be added, removed, or reconfigured without modifying static configuration files.
A further extension would be the introduction of actuator failure detection through periodic status messages, or heartbeat mechanisms. Comparing the requested actuator state with its reported state would allow failures to be detected and appropriate fallback actions to be taken.
Finally, the evaluation could be extended to larger deployments and more complex failure scenarios, including simultaneous sensor and actuator failures.